Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Group Details:

S.no	Name	Collage	Contact
1	MAKESH KUMAR D	KAAMADHENU ARTS & SCIENCE	makeshkumar.cs23@kascsathy.ac.in
2	MANIKANDAN R	KAAMADHENU ARTS & SCIENCE	manikandan.cs23@kascsathy.ac.in
3	MONILSON K	KAAMADHENU ARTS & SCIENCE	monilson.cs23@kascsathy.ac.in
4	KUMARAN B	KAAMADHENU ARTS & SCIENCE	kumaran.cs23@kascsathy.ac.in
5	SAKTHI SREE B	KAAMADHENU ARTS & SCIENCE	sakthisree.cs23@kascsathy.ac.in

Project Details:

Title: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Introduction:

As organizations increasingly adopt cloud services, remote work, and distributed applications, traditional network security models based on perimeter defences are no longer sufficient. Users, devices, and applications now operate outside the conventional corporate network, making it essential to adopt a more secure and flexible access model. This is where identity-aware, application-specific access policies become critical in ensuring that users only access the resources they truly need.

Zscaler Private Access (ZPA) is a cloud-delivered Zero Trust Network Access (ZTNA) solution designed to provide secure, direct access to private applications without exposing the internal network to the internet. Instead of granting broad network access through traditional VPNs, ZPA enforces least-privilege access, allowing users to connect only to specific applications based on verified identity, device posture, and contextual policies.

By leveraging identity providers, device compliance checks, and granular policy controls, ZPA ensures that access decisions are made dynamically and continuously. This approach minimizes the attack surface, prevents lateral movement within the network, and strengthens overall security posture. Implementing least-privilege access through ZPA enables organizations to protect sensitive applications while delivering seamless and secure connectivity for modern, distributed workforces.

Ultimately, this project will enforce a least-privilege access model, ensuring users and devices are granted the minimum required access necessary to perform their functions. By decoupling application access from network access, we will eliminate the risk of lateral movement across the network, as users have no visibility or connectivity to other hosts. This strategic move not only significantly enhances security by reducing the attack surface but also simplifies the user experience and reduces operational overhead associated with legacy remote access infrastructure, positioning the organization at the forefront of secure and efficient access management.

Abstract:

Implementing secure access to enterprise applications has become increasingly important as organizations shift toward cloud-based environments and remote work models. Traditional network security approaches, such as VPN-based access, often provide broad network connectivity, which increases the risk of unauthorized access and lateral movement within the network. To address these challenges, modern security frameworks emphasize the principle of least privilege and identity-aware access control.

Zscaler Private Access (ZPA) enables organizations to enforce application-specific access policies based on user identity, device posture, and contextual information. By adopting a Zero Trust approach, ZPA ensures that users are granted access only to the specific applications they are authorized to use, rather than the entire network. This reduces the attack surface and enhances protection for sensitive internal resources.

This study focuses on how least-privilege access can be implemented using identity-aware, application-centric policies through ZPA. It explores the architecture, policy enforcement mechanisms, and benefits of adopting this model, including improved security, reduced risk of data breaches, and simplified access management. The approach demonstrates how organizations can achieve secure, scalable, and efficient access to private applications while maintaining strict control over user privileges.

Use Case Scenarios:

1. Secure Remote Employee Access to Internal Applications Scenario:

A company has employees working remotely who need access to internal applications such as HR portals, internal dashboards, and development tools.

Implementation with ZPA:

- Users authenticate through an identity provider (e.g., SSO).
- Zscaler Private Access (ZPA) verifies user identity and device posture.
- Access policies allow employees to connect only to the specific applications required for their role, not the entire corporate network.

Outcome:

- Eliminates traditional VPN risks.
- Ensures **least-privilege access** for remote users.
- Prevents unauthorized access to other internal resources.

---

2. Third-Party Vendor Access to Specific Systems Scenario:

An organization hires an external vendor to maintain a database server or internal application.

Implementation with ZPA:

- Vendor accounts are created with limited permissions.
- ZPA policies restrict vendor access only to the maintenance application or server.
- Access is allowed only during predefined time windows.

Outcome:

- Vendors cannot see or access other corporate systems.
- Reduces risk of insider threats or accidental data exposure.

---

3. Developer Access to Development Environment Only Scenario:

Developers need access to development and testing environments but should not access production systems.

Implementation with ZPA:

- Identity-based policies map developers to Dev/Test applications only.
- Production environments are accessible only by authorized DevOps engineers.
- Device posture checks ensure secure developer machines.

Outcome:

- Prevents accidental changes to production systems.
- Supports secure DevOps practices with granular application-level access.

---

4. Secure Branch Office Application Access Scenario:

Employees in branch offices need access to centralized enterprise applications hosted in the data center or cloud.

Implementation with ZPA:

- Instead of exposing the network via VPN or MPLS, ZPA connects users directly to applications.
- Policies allow branch users to access specific business applications like ERP or CRM.

Outcome:

- Applications remain hidden from the public internet.
- Reduced network complexity and improved security.

---

5. BYOD (Bring Your Own Device) Secure Access Control Scenario:

Employees use personal devices to access company resources.

Implementation with ZPA:

- ZPA checks device compliance (OS version, security posture).
- Policies allow limited access to low-risk applications like internal portals.
- Sensitive systems are restricted to corporate-managed devices only.

Outcome:

- Secure access from personal devices.
- Protects critical applications and sensitive data.

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Stage-1

Overview:

Modern organizations require secure access to internal applications without exposing their entire network. Traditional security approaches, such as VPNs, often provide users with broad network access once they authenticate, which increases the risk of unauthorized access and lateral movement. Zscaler Private Access (ZPA), part of the Zscaler Zero Trust Exchange platform by Zscaler, addresses this problem by enforcing least-privilege access through identity-aware and application-specific policies.

The least-privilege principle ensures that users can only access the applications and resources necessary for their job roles. Instead of granting network-level access, ZPA connects users directly to specific applications. Access decisions are based on contextual factors such as user identity, device security status, location, and predefined organizational policies. This approach reduces the attack surface and prevents unauthorized users from accessing sensitive systems.

ZPA uses identity-aware access policies by integrating with identity providers like Microsoft Entra ID or Okta. When a user requests access to an application, ZPA verifies the user's identity and evaluates security conditions before allowing the connection. If the user meets the required criteria, a secure connection is established between the user and the application, while the internal network remains hidden from the internet.

Another key feature of ZPA is application-level segmentation. Instead of allowing access to an entire network, users can only reach the specific applications they are authorized to use. This prevents attackers from moving laterally within the network even if a device or account is compromised.

In addition, ZPA supports dynamic and context-aware policies that adapt to changing conditions. For example, access can be restricted if a device does not meet security requirements or if a login attempt occurs from an unusual location. This improves security while maintaining a smooth user experience.

Overall, Zscaler Private Access enables organizations to implement a Zero Trust security model, where access is granted based on identity, context, and policy rather than network location. By providing secure, application-specific access, ZPA helps organizations protect sensitive resources, reduce security risks, and ensure efficient remote access for users.

List of Teammates:

S.no	Name	Collage	Contact
1	MAKESH KUMAR D	KAAMADHENU ARTS & SCIENCE	makeshkumar.cs23@kascsathy.ac.in
2	MANIKANDAN R	KAAMADHENU ARTS & SCIENCE	manikandan.cs23@kascsathy.ac.in
3	MONILSON K	KAAMADHENU ARTS & SCIENCE	monilson.cs23@kascsathy.ac.in
4	KUMARAN B	KAAMADHENU ARTS & SCIENCE	kumaran.cs23@kascsathy.ac.in
5	SAKTHI SREE B	KAAMADHENU ARTS & SCIENCE	sakthisree.cs23@kascsathy.ac.in

List of Vulnerability Table:

Practice Website:

S.No	Vulnerability Name	CWE/CVE - No
1	Improper Authorization	CWE-285
2	Improper Restriction of XML External Entity Reference	CWE-611
3	Reliance on Insufficiently Trustworthy Component	CWE-1357
4	Key Exchange without Entity Authentication	CWE-322
5	SQL Injection: Hibernate	CWE-564
6	J2EE Bad Practices: Use of System. exit()	CWE-382

Report:

1.Vulnerability Name: - Improper Authorization

CWE/CVE: - CWE-285

OWASP/SANS Category: - A01:2025 Broken Access Control

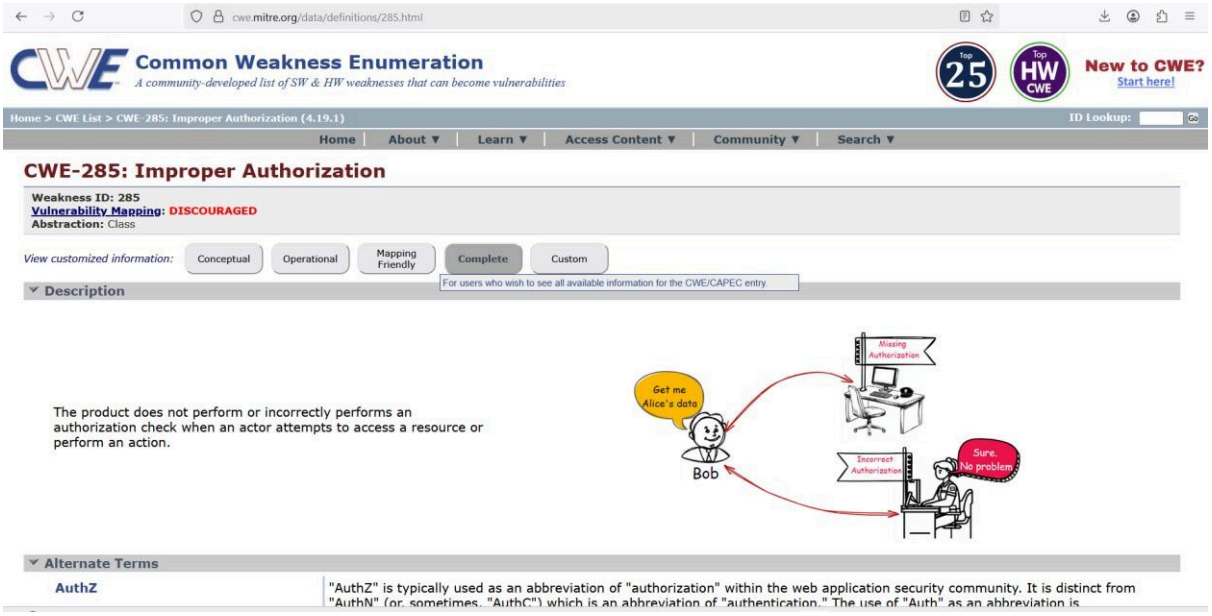
Description:

Improper Authorization occurs when an application fails to properly verify whether a user has permission to access specific resources or perform certain actions. Attackers can exploit this weakness to gain unauthorized access to sensitive data or restricted functions, potentially leading to privilege escalation and data breaches.

Business Impact:

Improper Authorization can allow unauthorized users to access sensitive data or perform restricted actions within an application. This may lead to data breaches, financial losses, reputational damage, and violation of regulatory compliance requirements. Additionally, attackers could escalate privileges and disrupt business operations or compromise critical systems.

Vulnerability path: https://cwe.mitre.org/data/definitions/285.html



2.Vulnerability Name: - Improper Restriction of XML External Entity Reference

CWE/CVE: - CWE-611

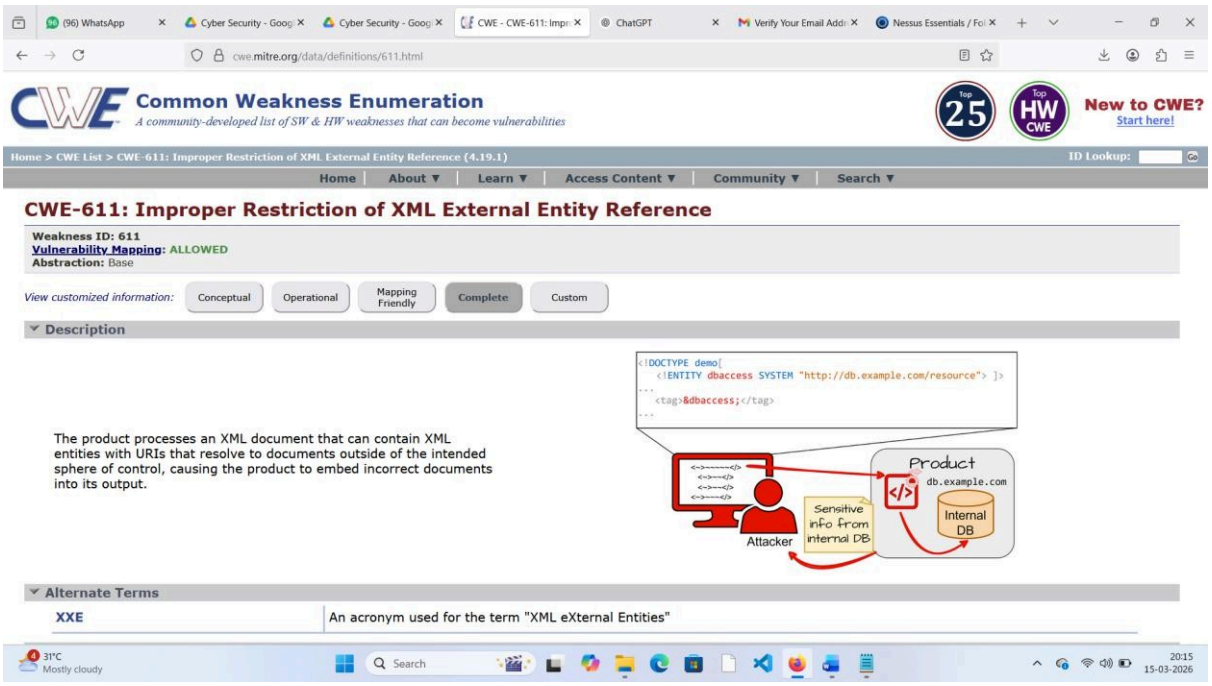
OWASP/SANS Category: - A02:2025 Security Misconfiguration

Description:

Improper Restriction of XML External Entity (XXE) occurs when an application processes XML input without properly restricting external entity references. Attackers can exploit this vulnerability by injecting malicious XML that references external resources. When the XML parser processes the input, it may retrieve sensitive files from the server, access internal systems, or trigger unintended network requests. This happens due to insecure XML parser configurations that allow external entities to be processed.

Business Impact:

This vulnerability can lead to sensitive data exposure, unauthorized access to internal files, and potential server-side request forgery (SSRF). Attackers may retrieve confidential information such as system files, credentials, or configuration data. In severe cases, it can compromise internal systems, disrupt services, and cause financial loss, reputational damage, and regulatory compliance violations.



3.Vulnerability Name: - Reliance on Insufficiently Trustworthy Component

CWE/CVE: - CWE-1357

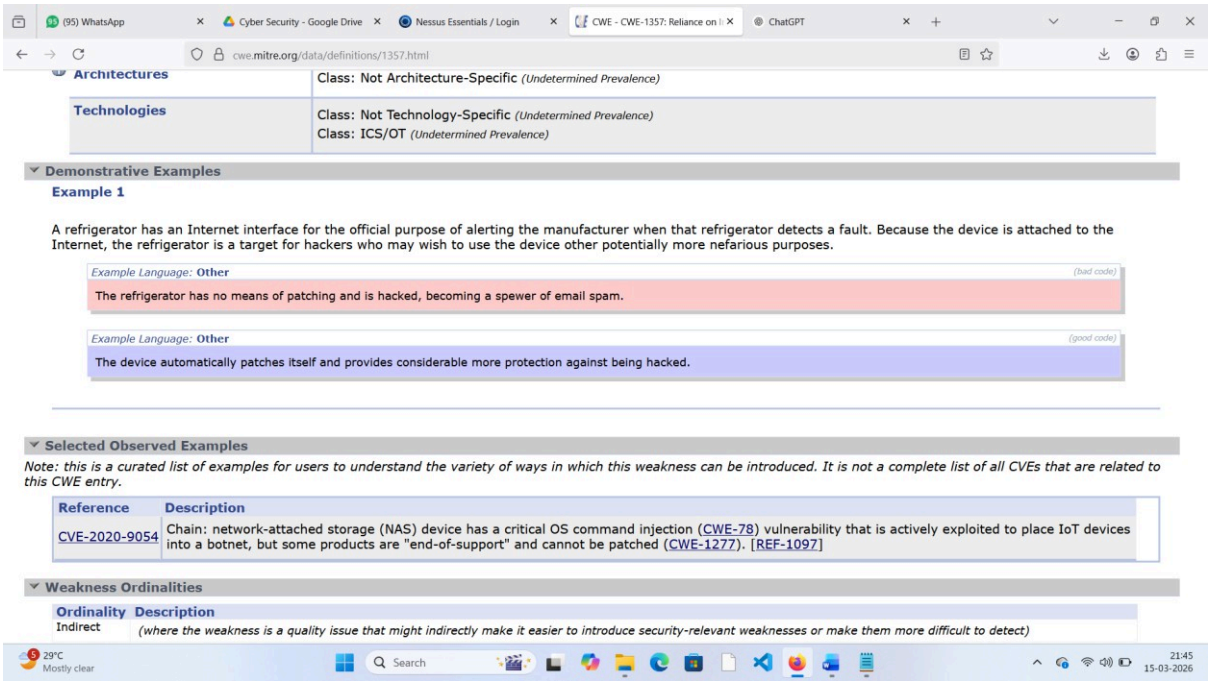
OWASP/SANS Category: - A03:2025 Software Supply Chain Failures

Description:

This vulnerability occurs when an application relies on third-party libraries, frameworks, or external components that are not properly verified or maintained. If these components contain vulnerabilities or malicious code, attackers may exploit them to compromise the application.

Business Impact:

Exploiting insecure components can lead to system compromise, data breaches, service disruption, and reputational damage, potentially resulting in financial loss and compliance issues.



Architectures	Class: Not Architecture-Specific *(Undetermined Prevalence)*
Technologies	Class: Not Technology-Specific *(Undetermined Prevalence)* Class: ICS/OT *(Undetermined Prevalence)*

Demonstrative Examples

Example 1

A refrigerator has an Internet interface for the official purpose of alerting the manufacturer when that refrigerator detects a fault. Because the device is attached to the Internet, the refrigerator is a target for hackers who may wish to use the device other potentially more nefarious purposes.

Example Language: **Other** *(bad code)*

The refrigerator has no means of patching and is hacked, becoming a spewer of email spam.

Example Language: **Other** *(good code)*

The device automatically patches itself and provides considerable more protection against being hacked.

Selected Observed Examples

Note: this is a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry.

Reference	Description
CVE-2020-9054	Chain: network-attached storage (NAS) device has a critical OS command injection (CWE-78) vulnerability that is actively exploited to place IoT devices into a botnet, but some products are "end-of-support" and cannot be patched (CWE-1277). [REF-1097]

Weakness Ordinalities

Ordinality	Description
Indirect	*(where the weakness is a quality issue that might indirectly make it easier to introduce security-relevant weaknesses or make them more difficult to detect)*

4.Vulnerability Name: Key Exchange without Entity Authentication

CWE/CVE: - CWE-322

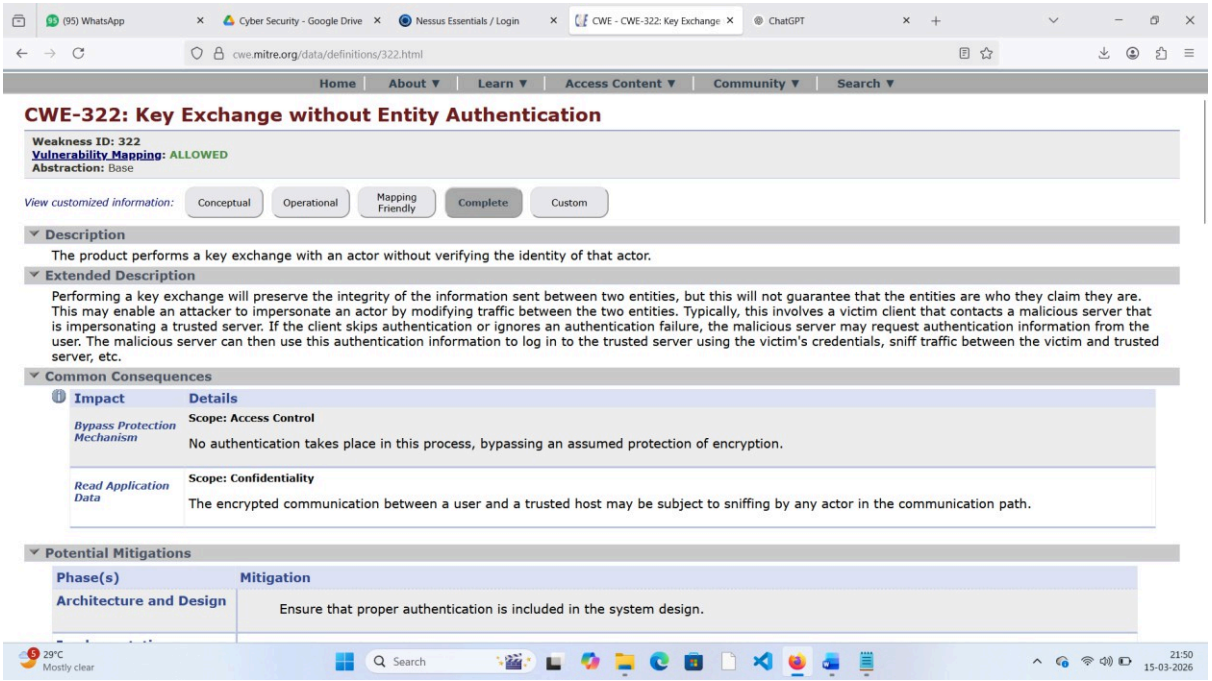
OWASP/SANS Category: - A04:2025 Cryptographic Failures

Description:

This vulnerability occurs when a cryptographic key exchange is performed without verifying the identity of the communicating parties. Without proper authentication, attackers can intercept or manipulate the key exchange process and perform a man-in-the-middle attack.

Business Impact:

Attackers may intercept sensitive data, alter communications, or gain unauthorized access to systems, leading to data breaches, loss of confidentiality, and potential reputational and financial damage to the organization.



(95) WhatsApp | Cyber Security - Google Drive | Nessus Essentials / Login | CWE - CWE-322: Key Exchange | ChatGPT

cwe.mitre.org/data/definitions/322.html

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-322: Key Exchange without Entity Authentication

Weakness ID: 322
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product performs a key exchange with an actor without verifying the identity of that actor.

Extended Description

Performing a key exchange will preserve the integrity of the information sent between two entities, but this will not guarantee that the entities are who they claim they are. This may enable an attacker to impersonate an actor by modifying traffic between the two entities. Typically, this involves a victim client that contacts a malicious server that is impersonating a trusted server. If the client skips authentication or ignores an authentication failure, the malicious server may request authentication information from the user. The malicious server can then use this authentication information to log in to the trusted server using the victim's credentials, sniff traffic between the victim and trusted server, etc.

Common Consequences

Impact	Details
Bypass Protection Mechanism	**Scope: Access Control** No authentication takes place in this process, bypassing an assumed protection of encryption.
Read Application Data	**Scope: Confidentiality** The encrypted communication between a user and a trusted host may be subject to sniffing by any actor in the communication path.

Potential Mitigations

Phase(s)	Mitigation
Architecture and Design	Ensure that proper authentication is included in the system design.

29°C Mostly clear | Search | 21:50 15-03-2026

5.Vulnerability Name: SQL Injection: Hibernate

CWE/CVE: - CWE-564

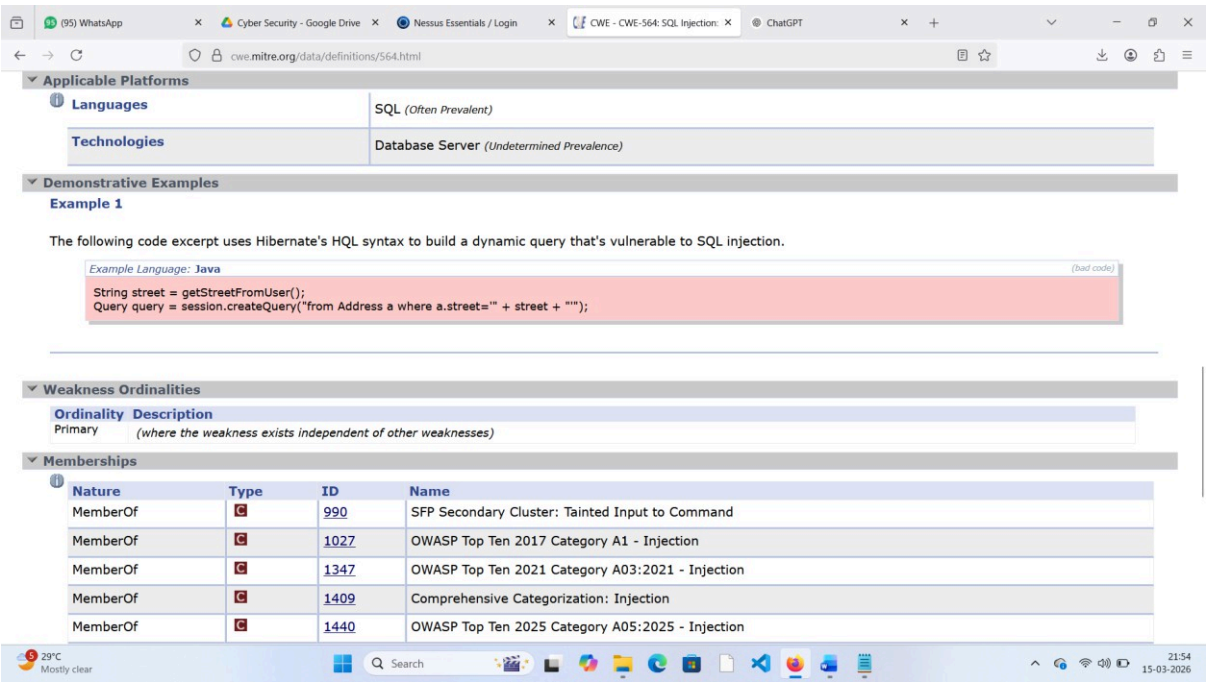
OWASP/SANS Category: - A05:2025 Injection

Description:

SQL Injection in Hibernate occurs when user-supplied input is directly incorporated into Hibernate Query Language (HQL) or native SQL queries without proper validation or parameterized queries. When developers dynamically build queries using concatenated input values, attackers can inject malicious SQL or HQL statements. This allows them to manipulate the intended database query executed by the application. Because Hibernate acts as an Object-Relational Mapping (ORM) framework, insecure query construction can bypass ORM protections and expose the underlying database to injection attacks. An attacker may exploit this vulnerability to retrieve sensitive data, modify database records, bypass authentication mechanisms, or execute administrative database operations.

Business Impact:

Successful exploitation of this vulnerability can have serious consequences for an organization. Attackers may gain unauthorized access to sensitive information such as user credentials, financial data, or confidential business records. They may also alter or delete critical database information, which can disrupt business operations and compromise data integrity. Additionally, SQL injection attacks can lead to complete database compromise, service disruption, regulatory compliance violations, financial losses, and significant reputational damage if customer or organizational data is exposed.



6.Vulnerability Name: J2EE Bad Practices: Use of System. exit()

CWE/CVE: - CWE-382

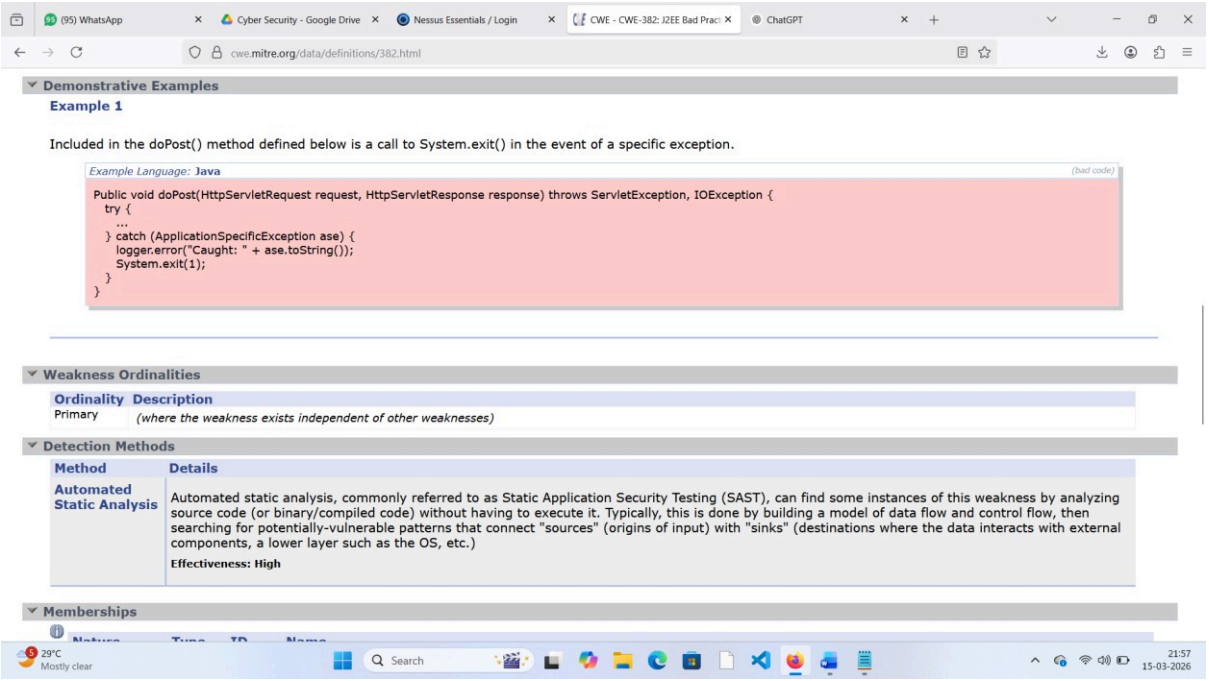
OWASP/SANS Category: - A06:2025 Insecure Design

Description:

This vulnerability occurs when a J2EE application uses System. Exit() to terminate the Java Virtual Machine (JVM). In enterprise environments such as application servers, calling System. Exit() can abruptly stop the entire JVM, affecting all running applications and services. Improper use of this method can lead to unexpected application shutdowns and disrupt normal system operations.

Business Impact:

Exploitation or unintended execution may cause application crashes or service outages, leading to disruption of business operations. It can affect system availability, impact multiple services running on the same server, and potentially result in financial loss and reduced user trust.



Stage 2

Overview:

Vulnerability Scanning using Nessus:

In this stage, vulnerability assessment was performed using Nessus Essentials, a widely used vulnerability scanning tool developed by Tenable Inc. Nessus is used by security professionals to identify vulnerabilities, misconfigurations, and security weaknesses in networks, systems, and web applications.

Nessus operates using a plugin-based vulnerability detection system, where each plugin checks for a specific vulnerability or security issue. During a scan, Nessus sends multiple network probes to the target host and analyses the responses to determine whether vulnerabilities exist.

In this project, Nessus was installed locally and accessed through the web interface using the following URL:

https://localhost:8834

After the installation process, the Nessus plugins were compiled and updated to ensure accurate vulnerability detection. A Basic Network Scan was configured to analyse the target web server.

The target used for vulnerability scanning was testfire.net, which is a deliberately vulnerable web application designed for security testing and training purposes.

The scan analysed the server configuration, open ports, encryption protocols, and web server information. After the scan completed, Nessus generated a detailed vulnerability report containing several informational and medium-severity findings.

How Nessus Works

The vulnerability scanning process performed by Nessus follows several steps:

1. Host Discovery

Nessus begins by identifying whether the target system is reachable using protocols such as TCP, UDP, and ICMP. This step confirms that the system is active and available for scanning.

2. Service Detection

After confirming the host is active, Nessus identifies the open ports and services running on the target system. This includes identifying web servers, databases, application servers, and other services.

3. Vulnerability Detection

Nessus executes multiple vulnerability detection plugins against the discovered services. Each plugin checks for specific vulnerabilities such as outdated software, weak encryption protocols, or insecure configurations.

4. Vulnerability Analysis

The scanner analyses the responses from the target system to determine whether a vulnerability exists and assigns a severity level.

5. Report Generation

After completing the scan, Nessus generates a comprehensive report that includes:

• Vulnerability name

• Severity level

• Plugin identification number

• Description of the vulnerability

• Remediation recommendations

Benefits of Using Nessus Nessus

Nessus provides several advantages for vulnerability assessment:

● **Comprehensive vulnerability coverage** – Nessus contains thousands of plugins capable of detecting a wide range of vulnerabilities across systems and applications.

● **Ease of use** – The Nessus web interface allows users to easily configure and launch vulnerability scans.

● **Detailed reporting** – Nessus generates detailed reports that provide technical descriptions and remediation recommendations for each vulnerability.

● **Scalability** – Nessus can scan individual hosts, entire networks, or web applications.

● **Flexible scanning options** – Scans can be executed manually or scheduled periodically for continuous monitoring.

Due to these capabilities, Nessus is widely used by security professionals for vulnerability assessment and penetration testing.

Target Website

http://testfire.net

Target IP Address

65.61.137.117

The website **testfire.net** is a purposely vulnerable web application designed for security testing and training purposes. It allows security professionals and students to perform vulnerability scans safely without affecting real production systems.

List of vulnerability:-

S.no	Vulnerability name	Severity	plugins
1	TLS Version 1.1 Protocol Deprecated	MEDIUM	157288
2	SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)	LOW	83875
3	HTTP Multiple Issues	INFO	10107
4	SSL Multiple Issues	INFO	51192
5	TLS Multiple Issues	INFO	42873
6	IETF MD5 Multiple Issues	INFO	35291
7	Service Detection	INFO	22964
8	Apache Tomcat Detection	INFO	39446
9	Nessus SYN Scanner	INFO	11219
10	Additional DNS Hostnames	INFO	46180

REPORT:-

1) Vulnerability Name: - 157288 - TLS Version 1.1 Protocol Deprecated

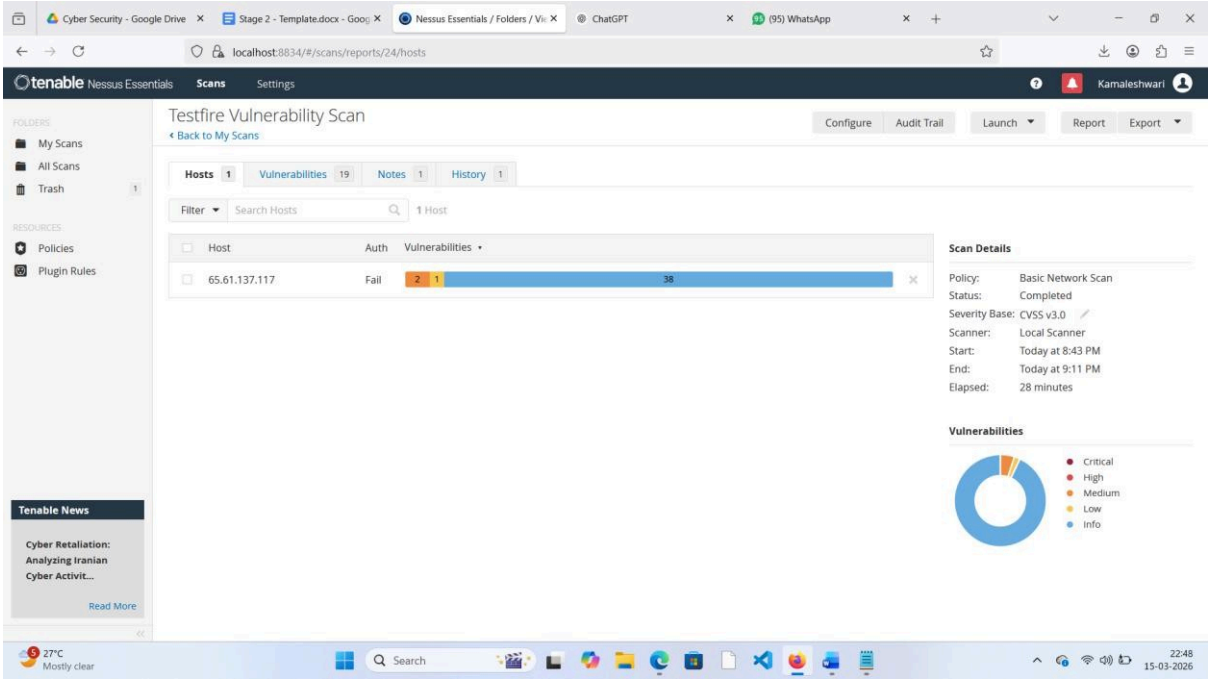
Severity: - MEDIUM **Plugin: -** 157288

Port: - 443

Description: -

The remote service accepts connections encrypted using TLS 1.1 protocol. TLS 1.1 lacks support for modern encryption algorithms and recommended cipher suites. Ciphers that support encryption before MAC computation and authenticated encryption modes such as GCM cannot be used with TLS 1.1.

As of March 31, 2020, endpoints that are not enabled for TLS 1.2 or higher will no longer function properly with major web browsers and vendors.



"TLS Version 1.1 Deprecated Protocol – Plugin ID 157288"

Solution: -

Enable support for TLS 1.2 and TLS 1.3 and disable support for TLS 1.1.

Business Impact: -

Using outdated encryption protocols can expose systems to cryptographic attacks and data interception. It may also cause compatibility issues with modern browsers and reduce the overall security of the server.

2) Vulnerability Name: - 83875 - SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

Severity: - LOW **Plugin: -** 83875

Port: - 443

Description: -

The remote host allows SSL/TLS connections with Diffie-Hellman parameters that are less than or equal to 1024 bits. Weak Diffie-Hellman parameters may allow attackers to perform cryptographic attacks such as the Logjam attack.

Solution: -

Reconfigure the server to use Diffie-Hellman parameters of 2048 bits or greater.

Business Impact: -

Weak encryption parameters may allow attackers to compromise encrypted communications and potentially recover sensitive data transmitted over the network.

3) Vulnerability Name: - 10107 - HTTP Multiple Issues

Severity: - INFO **Plugin: -** 10107

Port: - 80

Description: -

This vulnerability indicates that the server has multiple informational issues related to the HTTP service configuration.

Solution: - Review HTTP server configurations and ensure that unnecessary information is not exposed to users or attackers.

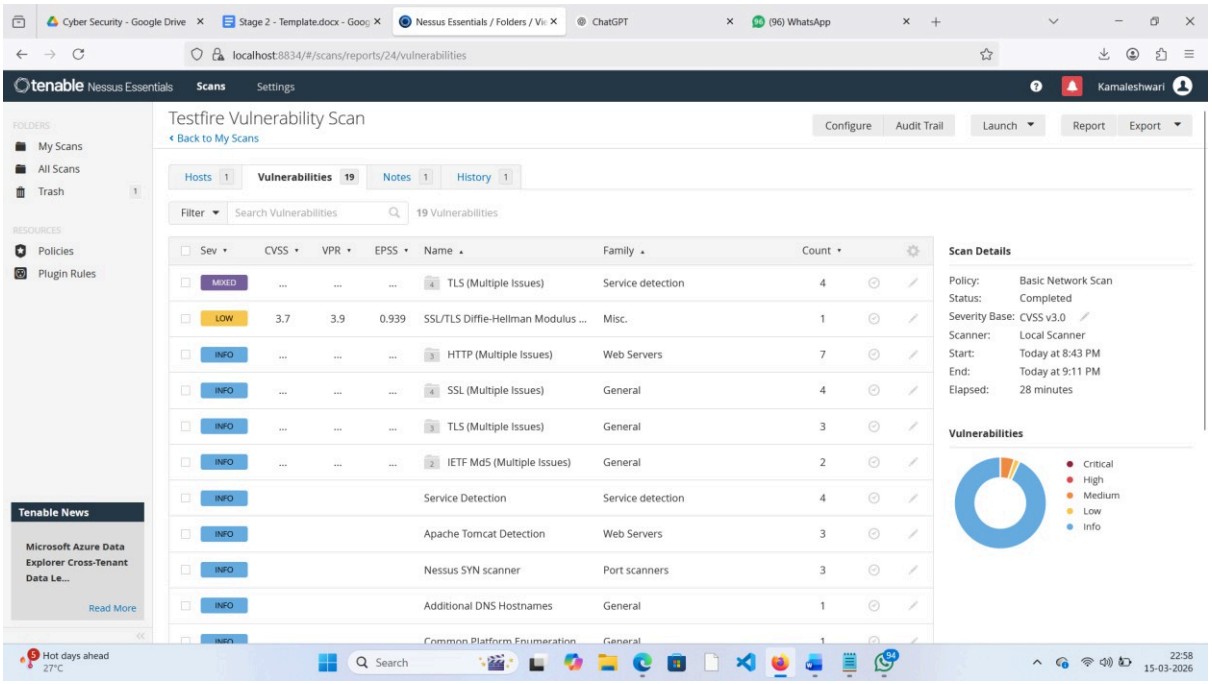
Business Impact: - While this issue does not directly affect system functionality, exposing server information may help attackers identify potential weaknesses in the system.

4) Vulnerability Name: - 51192 - SSL Multiple Issues

Severity: - INFO **Plugin: -** 51192

Port: - 443

Description: - The Nessus scan detected several informational findings related to the SSL configuration of the remote server.



Solution: -

Review the SSL configuration and ensure that recommended security standards are followed.

Business Impact: -

Improper SSL configuration can weaken the encryption security and may expose the server to potential attacks.

5) Vulnerability Name: - 42873 - TLS Multiple Issues

Severity: - INFO **Plugin: -** 42873

Port: - 443

Description: -

The scan detected multiple TLS-related informational findings related to encryption configuration.

Solution: -

Ensure that only secure TLS versions and cipher suites are enabled.

Business Impact: -

Improper TLS configuration may reduce the overall security of encrypted communications.

6) Vulnerability Name: - 35291 - IETF MD5 Multiple Issues

Severity: - INFO **Plugin**: - 35291

Port: - 0

Description: - The scan detected MD5 related cryptographic components. MD5 is considered a weak hashing algorithm and should not be used for security-sensitive operations.

Solution: - Replace MD5 with stronger hashing algorithms such as SHA-256 or SHA-512.

Business Impact: - Weak hashing algorithms may allow attackers to compromise data integrity and authentication mechanisms.

7) Vulnerability Name: - 22964 - Service Detection

Severity: - INFO **Plugin: -** 22964

Port: - 0

Description: - This plugin identifies services running on the remote host.

Solution: - Administrators should review exposed services and ensure that unnecessary services are disabled.

Business Impact: - Service discovery information may assist attackers in identifying potential attack vectors.

8) Vulnerability Name: - 39446 - Apache Tomcat Detection

Severity: - INFO **Plugin: -** 39446

Port: - 80

Description: - Nessus detected that the server is running Apache Tomcat, which is a Java based web application server.

Solution: - Ensure that Apache Tomcat is updated and properly configured with secure access controls.

Business Impact: - Outdated versions of Apache Tomcat may contain vulnerabilities that attackers could exploit.

9) Vulnerability Name: - 11219 - Nessus SYN Scanner

Severity: - INFO **Plugin: -** 11219

Port: - 0

Description: - This plugin performs SYN scanning to identify open ports on the target system.

Solution: - Administrators should review open ports and disable unnecessary services.
Business Impact: - Open ports may expose services that could be targeted by attackers.

10) Vulnerability Name: - 46180 - Additional DNS Hostnames

Severity: - INFO **Plugin: -** 46180

Port: - 0

Description: - The Nessus scan detected additional DNS hostnames associated with the target server.

Solution: - Review DNS configurations and ensure that only required hostnames are exposed.

Business Impact: This relates to the identification of the remote web server's type and version. While this issue doesn't pose an immediate security threat, it can provide valuable information to potential attackers about the server's software stack. Knowledge of the server type and version may aid attackers in exploiting known vulnerabilities specific to that software, making it important to restrict such information from being easily accessible to enhance the server's overall security posture.

Stage-3

Title:

Enforcing Least-Privilege Access through Identity-Aware, Application-Specific Access Policies using Zscaler Private Access (ZPA)

Report

1. SOC (Security Operations Center)

A **Security Operations Center (SOC)** is a centralized unit responsible for continuously monitoring, detecting, analyzing, and responding to cybersecurity threats within an organization. The SOC team uses advanced security tools, threat intelligence, and monitoring systems to identify suspicious activities across networks, servers, applications, and endpoints. SOC analysts work in shifts to provide **24/7 security monitoring** and ensure quick response to potential attacks. By implementing solutions such as **Zscaler Private Access (ZPA)**, SOC teams can enforce **least-privilege access**, ensuring users can only access specific applications they are authorized to use. This improves overall network security and reduces the attack surface.

2. SOC Cycle

The **SOC cycle** describes the continuous process used by SOC teams to manage security operations. It typically includes the following stages:

1. **Monitoring:** Continuous observation of network traffic, system logs, and user activities.
2. **Detection:** Identifying suspicious or malicious behavior using tools like SIEM and threat intelligence feeds.
3. **Investigation:** Security analysts analyze alerts to determine whether they represent real threats.
4. **Response:** Appropriate actions are taken to mitigate or contain the threat.
5. **Recovery:** Systems are restored and security gaps are fixed.
6. **Improvement:** Lessons learned are used to enhance security controls and prevent future attacks.

This cycle helps organizations maintain strong security posture and quickly respond to cyber threats.

3. SIEM (Security Information and Event Management)

SIEM is a security solution that collects, analyzes, and correlates log data from multiple systems such as servers, firewalls, applications, and network devices. It helps security teams detect suspicious activities in real time. SIEM platforms combine **Security Information Management (SIM)** and **Security Event Management (SEM)** to provide centralized log monitoring and threat detection. In an organization using **Zscaler Private Access**, SIEM tools can collect access logs and analyze them to detect unauthorized access attempts or abnormal user behavior. This improves visibility and helps SOC analysts respond quickly to threats.

4. SIEM Cycle

The **SIEM cycle** represents the process of how SIEM tools collect and analyze security data.

1. **Data Collection:** Logs and security events are collected from multiple sources such as firewalls, servers, endpoints, and applications.
2. **Normalization:** Collected data is converted into a common format for easier analysis.
3. **Correlation:** SIEM analyzes events and identifies patterns that indicate potential security threats.
4. **Alerting:** If suspicious activity is detected, alerts are generated for SOC analysts.
5. **Investigation and Response:** Analysts investigate alerts and take necessary actions to mitigate risks.

This cycle helps organizations detect cyber threats faster and improve security monitoring.

5. MISP (Malware Information Sharing Platform)

MISP is an open-source platform used for **threat intelligence sharing** between organizations. It helps security teams share indicators of compromise (IOCs) such as malicious IP addresses, file hashes, and malware signatures. By sharing threat information, organizations can detect and prevent cyber attacks more effectively. MISP integrates with SIEM and SOC tools to automate threat intelligence updates. For example, if a malicious IP address is identified through MISP, the SOC team can quickly block it across the network using security controls like ZPA.

6. Your College Network Information

A typical **college network** includes multiple components such as student computer labs, faculty systems, administrative servers, Wi-Fi networks, and internet gateways. The network also supports learning management systems, email services, and research resources. Because

a large number of users access the network, it becomes an attractive target for cyber attackers. Implementing **identity-based access controls** and monitoring systems helps secure the network. Using solutions like **Zscaler Private Access**, students and staff can securely access internal applications without exposing the network directly to the internet.

7. Deploying SOC in Your College

To deploy a **SOC in a college environment**, the institution would first establish a centralized security monitoring team. Security tools such as **SIEM, endpoint detection systems, firewalls, and threat intelligence platforms** would be integrated into the SOC. The SOC would monitor network traffic, detect suspicious activities, and respond to security incidents affecting student systems, faculty systems, and servers. By integrating **Zscaler Private Access**, the college can enforce least-privilege access policies so that users only access specific applications they require, improving overall security and reducing unauthorized access risks.

8. Threat Intelligence

Threat intelligence refers to information about current and emerging cyber threats that can help organizations identify and prevent attacks. It includes data such as attacker techniques, malicious IP addresses, malware signatures, and vulnerabilities. Threat intelligence can come from internal sources, security vendors, and intelligence-sharing platforms like MISP. When integrated with SIEM systems, threat intelligence allows SOC teams to detect known threats faster and take preventive actions. This proactive approach improves security posture and helps organizations stay ahead of cyber attackers.

9. Incident Response

Incident response is the structured process used by organizations to detect, analyze, and recover from cybersecurity incidents. It usually follows several stages:

1. **Preparation:** Establish security policies and response plans.
2. **Detection and Analysis:** Identify potential security incidents.
3. **Containment:** Limit the impact of the attack.
4. **Eradication:** Remove the threat from affected systems.
5. **Recovery:** Restore systems and services to normal operations.
6. **Post-Incident Review:** Analyze the incident and improve defenses.

A strong incident response strategy helps minimize damage and ensures faster recovery from cyber attacks.

---

10. QRadar & Understanding the Tool

IBM QRadar is a widely used **SIEM platform** that helps organizations detect and respond to cyber threats. It collects log data from various sources such as network devices, servers, and applications. QRadar uses advanced analytics and correlation rules to identify suspicious activities. The dashboard provides visual insights into security events, enabling SOC analysts to quickly detect anomalies and investigate incidents. QRadar also integrates with threat intelligence platforms to enhance detection capabilities. Understanding how to use QRadar dashboards helps security teams efficiently monitor networks and manage security incidents.

Conclusion

Stage 1 – Web Application Testing

From **web application testing**, we understood how to identify vulnerabilities in web applications such as SQL injection, authentication flaws, and insecure configurations. Testing helps ensure that web applications are secure before deployment.

Stage 2 – Nessus Report

From the **Nessus vulnerability scanning report**, we learned how automated tools detect vulnerabilities in systems and networks. Nessus provides detailed reports that help organizations identify security weaknesses and prioritize remediation.

Stage 3 – SOC / SIEM / QRadar Dashboard

From studying **SOC, SIEM, and QRadar dashboards**, we understood how organizations monitor security events in real time. These tools help detect threats, analyze security incidents, and support quick response to cyber attacks.

Future Scope

Stage 1 – Future Scope of Web Application Testing

Web application testing will continue to evolve with the growth of cloud computing and microservices. Advanced testing techniques such as automated security testing, DevSecOps integration, and AI-based vulnerability detection will improve application security.

Stage 2 – Future Scope of Testing Process

The future of security testing will involve more automation and integration with development pipelines. Continuous security testing and advanced scanning tools will help organizations detect vulnerabilities earlier in the software development lifecycle.

Stage 3 – Future Scope of SOC / SIEM

SOC and SIEM technologies will become more intelligent with **AI-driven threat detection, automated incident response, and advanced threat intelligence integration**. These improvements will help organizations detect cyber threats faster and respond more efficiently.

Topics Explored

- Security Operations Center (SOC)
- SOC Operational Cycle
- SIEM Concepts and SIEM Cycle
- Threat Intelligence and MISP
- Incident Response Process
- Network Security Monitoring
- Identity-based Access Control using ZPA
- Security Monitoring using QRadar

Tools Explored

- **IBM QRadar (SIEM Platform)**
- **Nessus Vulnerability Scanner**
- **Zscaler Private Access (ZPA)**
- **MISP Threat Intelligence Platform**